Cheatsheet · 3 chapters on one page

TestNG cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

18 key ideas | 0 formulas | 6 tables | 13 tips and traps

STEP 1 TestNG Framework

1.1 TestNG Foundations: Annotations, Assertions and Test Control

A main method can run checks, but it cannot report which ones passed, skip the rest when setup fails, or rerun only the failures. TestNG does all of that through annotations. Every example on this page runs on real TestNG, and the same idea is shown in pytest and Mocha, so you can compare the three most common test runners for testers.

KEY IDEAS

- **Why a test framework** Test classes go in src/test/java . Run them from the IDE (right click, Run), from a testng.xml file, or with mvn test , where the Maven Surefire plugin finds the tests.
- **A first test class** A test method is a public void method marked @Test . It calls the code under test and checks the result with an assertion . If every assertion holds, the test passes.
- **The annotation lifecycle**
- **Hard and soft assertions**
- **Test attributes** Without priorities, TestNG runs test methods of a class in alphabetical order , not in the order they are written. Tests should not depend on that order. When order matters, say so with priority or dependsOnMethods .
- **Dependencies between tests** dependsOnMethods makes a test wait for others and run only if they passed. If a dependency fails, the dependent test is skipped , not failed, so the report shows one real failure instead of a cascade of misleading ones.

TABLES

Table: need · plain main method · testng

Need	Plain main method	TestNG
Know which checks passed	Read the printed lines yourself	Pass, fail and skip counts, HTML and XML reports
Keep going after one failure	The first exception stops everything	Each @Test is independent
Set up and clean up	Copy the code into every check	@BeforeMethod, @AfterClass and the rest
Choose what to run	Comment code in and out	testng.xml, groups, include and exclude
Run with different data	Loops written by hand	@DataProvider and @Parameters
Rerun only failures	Not possible	testng-failed.xml, retry analyzers

Lesson: Why a test framework

Table: annotation · runs · typical use in ui automation

Annotation	Runs	Typical use in UI automation
@BeforeSuite / @AfterSuite	Once for the whole run	Start and close the report, load configuration
@BeforeTest / @AfterTest	Once per `<test>` tag in testng.xml	Set the browser for that block of tests
@BeforeClass / @AfterClass	Once per test class	Launch and quit the browser
@BeforeMethod / @AfterMethod	Before and after every @Test	Open the start page, clear cookies, take a screenshot on failure

Lesson: The annotation lifecycle

Table: attribute · effect

Attribute	Effect
`priority = 1`	Lower numbers run first. The default is 0, and negative values are allowed
`enabled = false`	The test is not run at all and not counted
`description = "…"`	Text shown in the reports
`timeOut = 200`	Fails the test if it takes longer than 200 milliseconds
`invocationCount = 3`	Runs the same test 3 times (add `threadPoolSize` to run them in parallel)
`expectedExceptions = X.class`	The test passes only if X is thrown. With `expectedExceptionsMessageRegExp`, the message must match too

Lesson: Test attributes

TIPS AND TRAPS

REMEMBER — The examples here run on TestNG 6.9, which is what this site's code runner has. Everything shown works the same in TestNG 7, the version to use in new projects. In pytest the same roles are played by test functions and fixtures, in Mocha by `describe`, `it` and hooks.

Lesson: Why a test framework

REMEMBER — TestNG's `Assert.assertEquals(actual, expected)` takes the actual value **first**. JUnit uses the opposite order. Swapping them still passes and fails correctly, but the failure message then says "expected 3 but found 4", which sends you looking in the wrong place.

Lesson: A first test class

REMEMBER — pytest builds the same layers with fixture scopes: `session`, `module`, `class` and `function`. The code before `yield` is the setup, the code after it the teardown. Mocha has `before`, `beforeEach`, `afterEach` and `after`.

Lesson: The annotation lifecycle

COMMON MISTAKE — Forgetting `soft.assertAll()` is a classic bug: every check fails silently and the test still passes. Create a new SoftAssert inside each test, never share one between tests.

Lesson: Hard and soft assertions

IN THE EXAM — Prefer independent tests. Each one should prepare its own state, for example by logging in through an API in @BeforeMethod. Dependencies make sense for real end to end flows, where a later step truly cannot start without the earlier one.

Lesson: Dependencies between tests

1.2 Suites, Groups, Parameters and Data Providers

Once a project has hundreds of tests, the questions change: which tests run tonight, on which browser, with which data, and how fast. TestNG answers them outside the test code, in testng.xml, groups, parameters and data providers. Every

Java example on this page runs on real TestNG with the suite file shown beside it.

KEY IDEAS

- **The testng.xml suite file** testng.xml describes a run without touching the test code: which classes, which methods, in which <test> blocks. One suite holds one or more tests , each test holds classes, and each class can include or exclude methods.
- **Groups for smoke and regression runs** A test can belong to several groups : @Test(groups = {"smoke", "login"}) . The suite file then runs a slice of the project, such as every smoke test except the ones known to be broken. Exclude always wins over include.
- **Parameters from testng.xml** @Parameters({"browser", "baseUrl"}) fills method arguments from <parameter> tags. A parameter in a <test> overrides the same name at suite level, so one suite can run the same class on Chrome and Firefox.
- **Data providers** A data provider is a method that returns rows of arguments. TestNG calls the test once per row and reports each run with its arguments, so a failure names the exact data that broke it.
- **Data providers that read files** A provider can return an Iterator<Object[]> instead of an array, so rows are produced one at a time. It can also take a Method argument to know which test is asking, and live in a separate class shared through dataProviderClass .
- **Parallel execution**

TABLES

Table: parallel= · what runs at the same time

parallel=	What runs at the same time
"methods"	Test methods, even from the same class
"classes"	Whole classes, methods inside a class stay sequential
"tests"	Each `<test>` block, for example one per browser
"instances"	Instances of the same class, useful with @Factory

Lesson: Parallel execution

TIPS AND TRAPS

REMEMBER

Classes are named with their package, for example `com.shop.tests.LoginTests`. `<packages> <package name="com.shop.tests"/></packages>` runs every class in a package. With Maven, point the Surefire plugin at the file with `<suiteXmlFile>testng.xml</suiteXmlFile>`.

Lesson: The testng.xml suite file

COMMON MISTAKE

Configuration methods such as @BeforeMethod and @BeforeGroups belong to no group by default, so a group filter leaves them out. Add `alwaysRun = true` to setup methods, or give them the same groups, or your tests will run without their setup.

Lesson: Groups for smoke and regression runs

REMEMBER

pytest's `@pytest.mark.parametrize` is the same idea, and each row appears in the report as `test_login[admin-wrong-invalid]`. Mocha has no built in equivalent, so tests loop over the rows and create one `it()` per row.

Lesson: Data providers

COMMON MISTAKE

Parallel tests must not share changeable state. A plain `int` counter or a single `WebDriver` field shared by threads gives random failures. Use `AtomicInteger` and concurrent collections for counters, and `ThreadLocal<WebDriver>` so each thread drives its own browser.

Lesson: Parallel execution

1.3 Listeners, Retries, Factories and Framework Design

A suite that runs every night needs more than assertions. It needs a screenshot when something fails, a second chance for a timeout on a slow server, the same checks for every language or role, and a clean structure so that one change in the login page touches one file. This page builds those pieces on real TestNG.

KEY IDEAS

- **Listeners** A listener is a class TestNG calls on every test event: start, success, failure, skip, and the start and end of each <test> .
- **Retrying flaky tests** An IRetryAnalyzer decides whether a failed test gets another attempt. It suits failures caused by the environment, such as a slow test server or a network blip. It never fixes a real bug: a wrong total fails on every attempt.
- **@Factory: one class, many instances** A data provider repeats one method with different arguments. A factory repeats a whole class: it returns several instances, each built with different constructor data, and every @Test of the class runs on every instance.
- **Conditional skips** Sometimes a test cannot run meaningfully: the payment sandbox is down, or a destructive test must never run outside the QA environment.
- **A framework: Page Objects and a BaseTest** In the Page Object Model , each page of the application gets a class that holds its locators and actions, such as loginAs(user, pass) . Tests talk to pages, never to locators. When a button's id changes, one page class changes and every test keeps working.
- **Reports, reruns and CI** A CI server such as Jenkins or GitHub Actions runs mvn test on every change or every night, publishes testng-results.xml as test results, and keeps the screenshots and HTML reports as build artifacts.

TABLES

Table: way to register · code

Way to register	Code
On a class	`@Listeners(ScreenshotListener.class)`
For the whole suite	`<listeners><listener class-name="com.shop.ScreenshotListener"/></listeners>` in testng.xml
Automatically	A `META-INF/services/org.testng.ITestNGListener` file listing the class

Lesson: Listeners

Table: file after a run · use

File after a run	Use
`emailable-report.html`	A single page summary to attach or email
`index.html`	The full browsable report
`testng-results.xml`	Machine readable results that CI servers and report tools import
`testng-failed.xml`	A ready suite with only the failed and skipped tests: run it to rerun the failures

Lesson: Reports, reruns and CI

TIPS AND TRAPS

REMEMBER

In a real suite, `onTestFailure` gets the driver from the test instance (`result.getInstance()`) and calls `((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE)`. In pytest the same job is done by the `pytest_runtest_makereport` hook in `conftest.py`, and in Mocha by an `afterEach` that checks `this.currentTest.state`.

Lesson: Listeners

COMMON MISTAKE

Retries hide problems if nobody looks at them. Count and report retried tests, keep the limit low (one or two), and treat a test that needs retries every night as a defect in the test or the environment.

Lesson: Retrying flaky tests

REMEMBER

`@Factory(dataProvider = "roles")` on a constructor combines both ideas: the provider supplies the rows and the factory builds one instance per row.

Lesson: @Factory: one class, many instances

IN THE EXAM

Keep assertions in tests and out of page objects. A page method returns what it sees, and the test decides whether that is right, so the same page can serve positive and negative tests.

Lesson: A framework: Page Objects and a BaseTest